

Image Classification Using CNNs

CS Elective 102 -- Coding Activities

Author: rbpelegrinojr

Generated: April 26, 2026

2 coding activities per lesson across all 20 lessons of the CNN Image Classification course. Each activity requires writing or running Python code using TensorFlow/Keras, NumPy, and related libraries.

Table of Contents

Lesson 01	What Is Image Classification?
Lesson 02	Understanding Digital Images & Pixels
Lesson 03	Introduction to Neural Networks
Lesson 04	From Dense Networks to CNNs
Lesson 05	The Convolution Operation Deep Dive
Lesson 06	Activation Functions
Lesson 07	Pooling Layers
Lesson 08	Building Your First Complete CNN
Lesson 09	Dataset Preparation & Loading
Lesson 10	Training a CNN
Lesson 11	Loss Functions & Optimizers
Lesson 12	Evaluating Your Model -- Metrics & Confusion Matrix
Lesson 13	Overfitting & Regularization Techniques
Lesson 14	Data Augmentation
Lesson 15	Batch Normalization & Dropout
Lesson 16	Transfer Learning
Lesson 17	Fine-Tuning Pretrained Models
Lesson 18	Model Visualization & Interpretability (Grad-CAM)
Lesson 19	Deploying Your CNN Model
Lesson 20	Capstone Project -- End-to-End Image Classifier

Lesson 01: What Is Image Classification?

Coding Activities

Load and Visualize MNIST: Using TensorFlow/Keras, load the MNIST dataset with `tf.keras.datasets.mnist.load_data()`. Print the shape and dtype of the training images. Then display a 2x5 grid (one image per digit class) using `matplotlib`.

Forward Pass Pipeline: Build a minimal Keras Sequential dense classifier for MNIST (Flatten -> Dense(128, relu) -> Dense(10, softmax)). Call `model.predict()` on one test image (add the batch dimension first) and print the resulting probability vector alongside the true label.

Lesson 02: Understanding Digital Images & Pixels

Coding Activities

RGB Channel Split: Load any color image with `PIL.Image.open()`. Convert it to a NumPy array and extract the R, G, and B channels separately. Display the three channels as grayscale images in a 1x3 matplotlib subplot.

Normalization Verification: Write a function `normalize_batch(images)` that accepts a uint8 NumPy array of shape (N, H, W, C) and returns a float32 array with values in [0.0, 1.0]. Apply it to the first 100 CIFAR-10 training images and use assert statements to verify the output dtype, min value, and max value.

Lesson 03: Introduction to Neural Networks

Coding Activities

Neuron from Scratch: In NumPy, implement a single ReLU neuron: given a weight vector w , bias b , and input x , compute $\text{ReLU}(w \cdot x + b)$. Test it with at least three different input vectors and print the outputs.

XOR MLP: Build a 2-layer keras.Sequential MLP (input: 2 -> Dense(4, relu) -> Dense(1, sigmoid)). Train it on the XOR dataset (4 samples) for 1 000 epochs with Adam. Print the final predictions and compare them to the true labels [0, 1, 1, 0].

Lesson 04: From Dense Networks to CNNs

Coding Activities

Parameter Count Comparison: Build (a) a dense network [3072 -> 2048 -> 1024 -> 10] and (b) a small CNN [Conv2D(32, 3) -> MaxPool -> Conv2D(64, 3) -> Flatten -> Dense(10)] for 32x32x3 inputs using the Keras Functional API. Call `model.summary()` on each and record the total parameter counts.

Feature Map Shape Tracing: Build a 2-block CNN in Keras and, for each Conv2D and MaxPooling2D layer, manually predict the output spatial dimensions using $\text{floor}((W - F + 2P) / S) + 1$. Verify your predictions match `model.summary()`.

Lesson 05: The Convolution Operation Deep Dive

Coding Activities

NumPy Convolution: Implement a `conv2d(image, kernel, stride=1)` function in pure NumPy using nested loops. Apply it to a 4x4 input with a 2x2 kernel and verify the result matches `scipy.signal.correlate2d(image, kernel, mode='valid')`.

Edge Detection with `tf.nn.conv2d`: Using `tf.nn.conv2d`, apply both a Sobel-horizontal and a Sobel-vertical kernel to a grayscale image. Display the original image and the two edge maps side by side in a 1x3 matplotlib subplot.

Lesson 06: Activation Functions

Coding Activities

Activation Function Plots: Write a script that uses NumPy and matplotlib to plot sigmoid, tanh, ReLU, and Leaky ReLU (alpha=0.1) on the range $x \in [-5, 5]$ in a single figure with a legend. Mark the derivative at $x=0$ for each function in the plot title.

Training Speed Comparison: Build two identical keras.Sequential networks for MNIST -- one using sigmoid activations throughout and one using ReLU. Train both for 5 epochs with Adam and plot both validation accuracy curves on the same axes. Record which converges faster.

Lesson 07: Pooling Layers

Coding Activities

MaxPool vs AveragePool: Using a `tf.keras.layers.MaxPooling2D` and a `tf.keras.layers.AveragePooling2D` layer (both `pool_size=2`, `stride=2`), apply each to the same randomly generated 8x8 feature map tensor. Print both output arrays and highlight cells where they differ.

Dimension Tracing: Build a small Keras CNN (2 Conv2D + MaxPooling2D blocks) for 32x32x3 inputs. Before running `model.summary()`, manually compute the spatial dimensions after each pooling layer. Verify your predictions against the summary output.

Lesson 08: Building Your First Complete CNN

Coding Activities

LeNet-5 in Keras: Implement LeNet-5 using `keras.Sequential` (`Conv2D(6,5,tanh)` -> `AvgPool` -> `Conv2D(16,5,tanh)` -> `AvgPool` -> `Flatten` -> `Dense(120,tanh)` -> `Dense(84,tanh)` -> `Dense(10,softmax)`). Run `model.summary()` and verify the total parameter count is approximately 61 706.

Architecture Modification: Start with the tiny 3-block CNN from `code/lesson_08.py`. Double the number of filters in every `Conv2D` layer. Compare total parameter count and validation accuracy after 3 epochs on MNIST to the original architecture.

Lesson 09: Dataset Preparation & Loading

Coding Activities

Dataset Exploration: Load CIFAR-10 with `tf.keras.datasets.cifar10.loaddata()`. Print the shape, dtype, min, max, and mean of the raw training images. Use `np.unique(ytrain, return_counts=True)` to confirm there are exactly 5 000 samples per class.

Pipeline Benchmarking: Build two `tf.data.Dataset` pipelines from the CIFAR-10 training data -- one without `.prefetch()` and one with `.prefetch(tf.data.AUTOTUNE)`. Use `time.time()` to measure the wall-clock time to iterate through one full epoch of batches (batch size 64) for each and print the difference.

Lesson 10: Training a CNN

Coding Activities

Manual Training Loop: In NumPy, implement a full training loop for a binary logistic regression problem: randomly initialize weights, compute sigmoid output, compute binary cross-entropy loss, compute gradients by hand, and update weights. Print the loss at every 100 epochs and confirm it decreases.

Keras model.fit() Training: Train the LeNet-5 model from Lesson 8 on MNIST using model.fit() with 10 epochs, batch size 64, a validation split of 0.1, and the Adam optimizer. After training, plot the training and validation accuracy curves on the same axes using matplotlib.

Lesson 11: Loss Functions & Optimizers

Coding Activities

Learning Rate Sweep: Train a small CNN on CIFAR-10 with learning rates [0.1, 0.01, 0.001, 0.0001]. Run each for 10 epochs with Adam. On a single matplotlib figure, plot the four training loss curves and label each with its learning rate.

Optimizer Comparison: Using the same CNN architecture and CIFAR-10, train three models: SGD(lr=0.01), SGD(lr=0.01, momentum=0.9), and Adam(lr=0.001). After 10 epochs, plot their validation accuracy curves on the same axes and print the final validation accuracy for each.

Lesson 12: Evaluating Your Model -- Metrics & Confusion Matrix

Coding Activities

Confusion Matrix: Train a CIFAR-10 classifier for 10 epochs. Generate predictions on the test set and plot the confusion matrix using `sklearn.metrics.ConfusionMatrixDisplay`. Identify and print the two most frequently confused class pairs.

Per-Class Metrics: Using `sklearn.metrics.classification_report`, compute per-class precision, recall, and F1-score for your CIFAR-10 model. Print the report and write code to extract and display the class with the lowest F1-score.

Lesson 13: Overfitting & Regularization Techniques

Coding Activities

Dropout vs No Dropout: Train a 4-layer dense network on 500 CIFAR-10 examples for 50 epochs -- once without any dropout and once with Dropout(0.5) after each hidden layer. On a single figure, plot training accuracy and validation accuracy for both runs to visualize the overfitting gap.

L2 Regularization Search: Train the same architecture with `kernel_regularizer=l2(lambda)` for `lambda_` in `[0, 1e-4, 1e-3, 1e-2]`. For each, record the best validation accuracy. Plot validation accuracy vs. `lambda` on a bar chart and print the best `lambda` value.

Lesson 14: Data Augmentation

Coding Activities

Augmentation Gallery: Pick one image from CIFAR-10. Apply each of these five transforms individually using Keras preprocessing layers: `RandomFlip('horizontal')`, `RandomRotation(0.2)`, `RandomZoom(0.2)`, `RandomContrast(0.3)`, and `RandomTranslation(0.1, 0.1)`. Display the original plus five augmented versions in a 2x3 matplotlib grid.

Augmentation Impact Study: Train two identical CNNs on CIFAR-10 for 15 epochs -- one with no augmentation and one with random horizontal flip, rotation ($\pm 15^\circ$), and zoom ($\pm 10\%$). Plot both validation accuracy curves on the same axes and print the final accuracy difference.

Lesson 15: Batch Normalization & Dropout

Coding Activities

Batch Normalization Stability: Build two identical 5-layer CNNs for CIFAR-10 -- one without any BatchNormalization layers and one with a BatchNormalization layer after every Conv2D. Train both for 10 epochs and plot their training loss curves on the same axes. Comment on the difference in stability.

Dropout Rate Search: Using the same CNN base, train five models with Dropout rates [0.0, 0.2, 0.4, 0.5, 0.7] inserted before the final Dense layer. After 10 epochs each, plot validation accuracy vs. dropout rate as a line graph and print the rate that gives the highest validation accuracy.

Lesson 16: Transfer Learning

Coding Activities

Feature Extractor Setup: Load `MobileNetV2(include_top=False, input_shape=(96,96,3), weights='imagenet')` and set `base_model.trainable = False`. Add `GlobalAveragePooling2D()` and `Dense(10, activation='softmax')` on top. Print `model.summary()` and confirm that no MobileNetV2 layers are marked as trainable.

Train the Classification Head: Using the frozen feature extractor from Activity 1, compile with Adam and categorical cross-entropy, and train on CIFAR-10 (images resized to 96x96, pixel values preprocessed with `tf.keras.applications.mobilenetv2.preprocess_input`) for 5 epochs. Print the final training and validation accuracy.

Lesson 17: Fine-Tuning Pretrained Models

Coding Activities

Two-Phase Training Curves: Starting from the frozen feature extractor (Phase 1) trained in Lesson 16, unfreeze the last 20 layers of MobileNetV2 and run Phase 2 fine-tuning for 10 epochs with Adam(lr=1e-5). Plot training and validation accuracy for both phases on a single graph, annotating the boundary between phases.

Learning Rate Sensitivity: Run three separate Phase 2 fine-tuning runs with learning rates [1e-3, 1e-4, 1e-5]. Plot the validation loss for all three on the same axes. Print which learning rate causes the validation loss to spike (catastrophic forgetting) and which converges most smoothly.

Lesson 18: Model Visualization & Interpretability (Grad-CAM)

Coding Activities

Grad-CAM Implementation: Implement Grad-CAM using `tf.GradientTape` targeting the last Conv2D layer of your CIFAR-10 model. Apply it to 5 correctly classified test images. For each, display the original image with the heat-map overlay (use `cv2.applyColorMap` or `matplotlib`'s `imshow` with alpha blending).

Feature Map Visualization: Build a Keras Model whose outputs are the activations of the first and last Conv2D layers in your trained network. Run a single test image through it. Display a 4x4 grid of feature maps from the first layer and another 4x4 grid from the last layer. Describe in a comment what visual patterns each layer captures.

Lesson 19: Deploying Your CNN Model

Coding Activities

Save and Reload: Save a trained Keras model in both .h5 format and SavedModel format using `model.save()`. Use `os.path.getsize()` (for .h5) and `os.path.getsize()` on each file in the SavedModel directory (sum them) to compare sizes. Reload each with `tf.keras.models.load_model()` and assert that predictions on 10 test images are identical.

TFLite Conversion and Timing: Convert the same Keras model to TFLite using `tf.lite.TFLiteConverter.from_keras_model()`. Load the .tflite file with `tf.lite.Interpreter`, run inference on a single image 100 times, and print the average inference time. Compare it to the average Keras inference time for the same image.

Lesson 20: Capstone Project -- End-to-End Image Classifier

Coding Activities

Run the Full Pipeline: Execute `code/lesson_20.py` end to end on the TensorFlow Flowers dataset. Record and print: final validation accuracy, total training time (Phase 1 + Phase 2), and model file size on disk.

Backbone Swap: Replace the MobileNetV2 backbone with `tf.keras.applications.EfficientNetB0`. Retrain the full two-phase pipeline on the same Flowers dataset. Compare and print: final validation accuracy, total parameter count (trainable vs. frozen), and average training time per epoch for both backbones side by side.